17.8

Helm & Package Management

Charts, values.yaml, Helmfile, Kustomize

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

What is Helm?

Helm is the package manager for Kubernetes. Instead of writing 10 separate YAML files for a deployment, service, ingress, etc., you use a Helm Chart — a template that generates all the YAML with your custom values. Install a whole application with one command.

■ **Simple Analogy:** Helm is to Kubernetes what apt/brew is to Linux/Mac. 'helm install postgres bitnami/postgresql' installs a fully configured PostgreSQL cluster with one command — like 'apt install postgresql' for K8s.

Helm Chart Structure

```
my-java-app/ # chart directory
Chart.yaml # chart metadata (name, version, description)
values.yaml # default values (overridable at install time)
charts/ # sub-charts (dependencies)
templates/
deployment.yaml # Go template: {{ .Values.replicaCount }} replicas
service.yaml
ingress.yaml
configmap.yaml
_helpers.tpl # reusable template snippets
NOTES.txt # post-install instructions shown to user
```

values.yaml — The Configuration File

```
# values.yaml (defaults)
replicaCount: 2
image:
repository: myregistry/java-api
tag: latest
pullPolicy: IfNotPresent
service:
type: ClusterIP
port: 80
resources:
limits: { cpu: 500m, memory: 512Mi }
requests: { cpu: 250m, memory: 256Mi }
ingress:
enabled: false
host: api.example.com
```

Common Helm Commands

```
helm install my-app ./my-java-app # install with defaults
helm install my-app ./my-java-app \
--set replicaCount=5 \ # override a value
--values prod-values.yaml \ # use prod overrides file
--namespace production

helm upgrade my-app ./my-java-app -f prod-values.yaml
helm rollback my-app 1 # roll back to revision 1
helm list -n production # list installed charts
helm uninstall my-app -n production # delete everything
```

Kustomize — Alternative to Helm

Kustomize is built into kubectl. Instead of templates, it uses overlays: a base YAML and environment-specific patches. No templating language needed — just YAML overlays.

```
base/ # base configuration (same across envs)
deployment.yaml
kustomization.yaml
overlays/
dev/
kustomization.yaml # patches: replicas=1, image=dev-tag
prod/
kustomization.yaml # patches: replicas=10, image=stable

kubectl apply -k overlays/prod/ # apply prod overlay
```

Helm vs Kustomize

Feature	Helm	Kustomize
Templating	Go templates (powerful)	YAML overlays (simpler)
Learning curve	Steeper	Gentler
Package sharing	Yes (Helm Hub/OCI)	No
Built into kubectl	No (separate install)	Yes
Best for	Reusable, shared charts	Env-specific patches

■ **Real-World Example:** Datadog distributes its monitoring agent as a Helm chart. Users run 'helm install datadog datadog/datadog --set datadog.apiKey=XXX' and get the full monitoring stack (DaemonSet, ClusterAgent, Service) instantly. Without Helm, this would require 15+ manual YAML files.